Overview

Version control is backbone of collaborative AI engineering. Experiments, feature engineering, model configs, and inference logic evolve rapidly; Git gives traceability, rollback, and safe collaboration.

Core Git Concepts

- **Repository:** versioned project workspace.
- **Commit:** immutable snapshot + message.
- **Branch:** independent line of changes.
- **Tag:** named release checkpoint.
- **Remote:** shared upstream repository.

Good commit messages should describe intent and risk, not only files touched.

Branching & Workflow

Common patterns: - **Feature branching:** branch per task, PR to `main`. - **Trunk-based:** short-lived branches, frequent integration. - **GitFlow:** structured `feature/release/hotfix` branches.

Tradeoff summary: - Trunk-based: faster integration, lower merge drift. - GitFlow: clearer release controls for regulated cycles.

Pull Requests & Code Review

PR workflow creates checkpoint for: - Correctness review. - Reproducibility verification. - Security and secret scanning. - Discussion of failure modes.

AI-specific review checklist: - Data leakage risk? - Reproducible seeds/config? - Metric claim backed by notebook output? - Backward compatibility for APIs/pipelines?

Practical AI Engineering Scenarios

Experiment Branch Strategy

Use branches like `exp/xgb-class-weight-v2` with clear scope. Keep experimental notebook noise isolated from production code.

PR for Pipeline Changes

When modifying feature pipelines, include: - Before/after schema samples. - Migration notes. - Rollback steps.

Common Pitfalls

- Committing large binary datasets into Git history.
- Mixing unrelated changes in one PR.
- Force-pushing shared branches without coordination.
- Rebasing incorrectly and dropping commits.

Business Case Studies & Exceptions

Case 1: Secret Leaked in Commit

API key committed in notebook output.

Response pattern: 1. Revoke key immediately. 2. Rotate credentials. 3. Remove from history using approved cleanup process. 4. Add scanning and pre-commit checks.

Case 2: Broken Merge Before Release

Model-serving branch merged without compatibility tests, causing runtime errors.

Mitigation: - Require CI status checks before merge. - Use protected branch rules. - Maintain release checklist with smoke tests.

Interview Questions & Answers

1. **Q: Why is Git crucial in ML projects?**
   A: It preserves experiment history, enables collaboration, and provides rollback for risky pipeline/model changes.
2. **Q: Feature branch vs trunk-based?**
   A: Feature branching isolates work longer; trunk-based promotes frequent integration and smaller merge conflicts.
3. **Q: What makes good PR for model change?**
   A: Clear problem statement, reproducible run details, metrics, risks, and rollback plan.
4. **Q: `merge` vs `rebase`?**
   A: Merge preserves branch history with merge commit; rebase rewrites commit base for linear history.
5. **Q: How handle merge conflict in notebook-heavy repo?**
   A: Prefer modular code in `.py`, clear ownership, and minimize simultaneous edits to same notebook.
6. **Q: Why avoid large data in Git?**
   A: Repository bloat hurts clone/pull performance and long-term maintainability.
7. **Q: What are protected branches?**
   A: Branch rules requiring checks/reviews before merge to prevent unsafe direct pushes.
8. **Q: How do you rollback bad release commit?**
   A: Create revert commit for faulty changes and redeploy previous stable artifact.
9. **Q: What commit granularity is ideal?**
   A: Small, cohesive commits with single intent; easier review, bisect, and rollback.
10. **Q: Why include CI in Git workflow?**
    A: Automated tests prevent regressions and enforce reproducibility before merge.